

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

“Д34: Технический дизайн микросервисной архитектуры”

Выполнил:

Преображенский Артемий

Группа:

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Разделить монолитное backend-приложение на микросервисы с соблюдением подхода database-per-service.
2. Спроектировать микросервисную архитектуру, описать взаимодействие между сервисами.
3. Разделить общую базу данных на несколько обособленных баз данных.
4. Спроектировать эндпоинты для межсервисного взаимодействия в формате OpenAPI.
5. Описать запросы, ответы и возможные ошибки.
6. Подготовить документ с требованиями и шагами по переходу от монолитного приложения к микросервисной архитектуре

Ход работы

Были выделены следующие микросервисы:

1. Auth Service — сервис пользователей и аутентификации.
Отвечает за регистрацию пользователя, вход в систему, выдачу JWT-токена и получение данных текущего пользователя.
В этот сервис были отнесены эндпоинты:
 - * /auth/register
 - * /auth/login
 - * /auth/me
2. Catalog Service — сервис каталога ресторанов.
Отвечает за получение списка ресторанов, просмотр информации о ресторане, меню, столиков и отзывов.
Также в этот сервис входят операции создания, изменения и удаления отзывов.
В этот сервис были отнесены эндпоинты:
 - * /restaurants
 - * /restaurants/{id}
 - * /restaurants/{id}/menu
 - * /restaurants/{id}/tables

- * /restaurants/{id}/reviews
- * /restaurants/{id}/reviews/{reviewID}

3. Booking Service — сервис бронирований.

Отвечает за проверку доступности столика, создание бронирования, просмотр списка бронирований пользователя, просмотр отдельного бронирования и отмену брони.

В этот сервис были отнесены эндпоинты:

- * /restaurants/{id}/tables/{tableID}/availability
- * /bookings
- * /me/bookings
- * /me/bookings/{bookingID}

В монолитной версии приложения связи между сущностями поддерживались внутри одной базы данных. В микросервисной архитектуре эти связи были перенесены на уровень HTTP-взаимодействия между сервисами. Например, сервис бронирований не обращается напрямую к таблицам ресторанов или пользователей, а получает необходимые данные через внутренние API других сервисов. Auth_db содержит таблицу users, catalog db содержит таблицы restaurants, menu_items, restaurant tables, reviews, booking_db содержит таблицу bookings.

Для межсервисного взаимодействия были спроектированы внутренние технические эндпоинты:

Auth Service

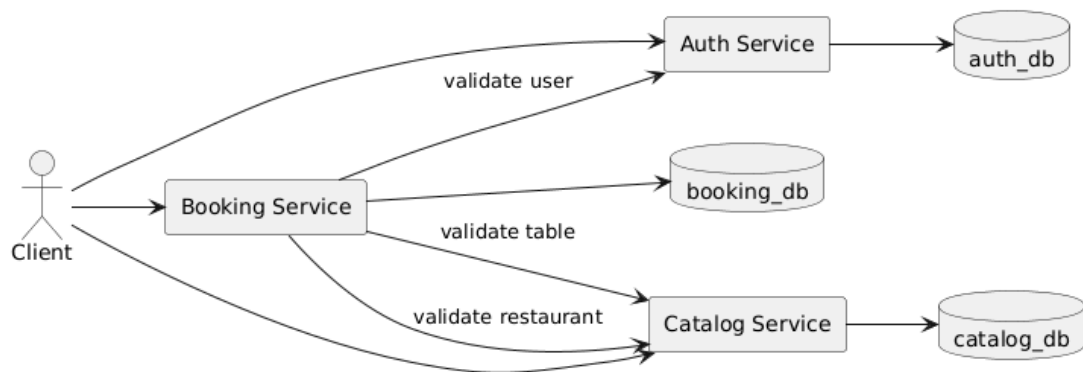
- * GET /internal/users/{id}

Catalog Service

- * GET /internal/restaurants/{id}
- * GET /internal/restaurants/{restaurantID}/tables/{tableID}

Пример логики взаимодействия сервисов при создании бронирования:

1. Клиент отправляет запрос в Booking Service.
2. Booking Service получает user_id из токена.
3. Booking Service обращается в Auth Service и проверяет существование пользователя.
4. Booking Service обращается в Catalog Service и проверяет существование ресторана.
5. Booking Service обращается в Catalog Service и проверяет существование столика и его принадлежность ресторану.
6. После успешной проверки создается запись о бронировании в booking_db.



Вывод

В ходе выполнения работы была спроектирована микросервисная архитектура для сервиса бронирования столиков в ресторанах. Система была разделена на три основных микросервиса: сервис аутентификации, сервис каталога ресторанов и сервис бронирований. Такое разделение лучше соответствует текущему API, уменьшает количество межсервисных зависимостей и позволяет сохранить архитектуру достаточно простой и понятной для дальнейшей реализации.